

Appendix - refreshing access tokens

What happens when an access code expires? Well, we need to refresh it, meaning that we need to get another access token from Keycloak. To do so, we need to employ the `refresh_token` we got when we logged in.

Access tokens have very short lifespans because they grant access to protected resources, whereas refresh tokens are longer-lived (Facebook, for example, uses 30-days-lived refresh tokens), because they are meant to be used multiple time over a session, to, in fact, refresh expired access tokens.

When refresh tokens expire, the user has to login again.

Refresh token usage

Let's imagine our API returned a `401 Unauthorized` error because our refresh token expired, using refresh tokens is quite simple and straightforward:

- assuming you saved your refresh token once you logged in and you have your client credentials, you must issue a `POST` request to the token endpoint, which is the same we [talked about in Chapter I](#);
- the request, though, has a different body:

```
{
  "client_id": "your client id",
  "client_secret": "your client secret",
  "refresh_token":
  "eyJhbGciOiJIUzUxMiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICI1NDVKNzUyNC1hMzg1LTRiOWItYjRhNy03ZTYwNDczYmU
  "grant_type": "refresh_token"
}
```

- as you can notice, we specify the type of request we are issuing using the `grant_type` parameter, which is set to a different value from the one we use when we are dealing with the standard flow (which is `authorization_code`).

If you want to try it yourself, there is an example request in the `oauth-2-spring-boot-auth.json` Postman collection:

